

BAZE DE DATE - PROIECT

SISTEM DE GESTIUNE A DATELOR PENTRU PLATFORMA VINTED

Student:

Șalaru Ioana

Grupa: 1309B

Coordonator:

ș.l. dr. ing. Mironeanu Cătălin

1. Scopul aplicației

Vinted este o aplicație pe care oricine o poate folosi pentru a vinde și cumpăra produse, precum îmbrăcăminte, electrocasnice mici, accesorii, piese de decor interior, jucării, etc., prețurile fiind de până la 2000 de lei. Evident, produsele sunt clasificate în categorii predefinite.

Avantajul pe care îl oferă în plus Vinted față de alte platforme de acest tip este că plățile sunt gestionate intermediar de administrația aplicației, astfel cumpărătorul efectuează plata la plasarea comenzii, iar banii ajung la vânzător atunci când cumpărătorul confirmă că produsele corespund.

- persoana poate să vândă și să cumpere produse; fiecare anunț se referă la un singur produs (când un articol este vândut, anunțul devine inactiv).
- Pentru un articol se pot preciza starea și o descriere, dar nu este obligatoriu.
- comanda se referă la un produs publicat de o persoană și cumpărat de o persoană.
- Doi utilizatori pot comunica prin mesaje.
- Prețul unui produs poate fi negociat, astfel o persoană poate oferi un preț, iar vânzătorul poate accepta prin actualizarea prețului produsului. Negocierile se realizează prin mesaje.
- Vom păstra o evidență a prețurilor anterioare ale unui produs.
- În urma unei comenzi, atât vânzătorul cât și cumpărătorul pot lăsa un feedback unul celuilalt, prin care primesc o notă de la 0–5 și justificarea acesteia. Acestea sunt publice și construiesc reputația contului.
- Având în vedere plățile online, conturile pot aparține doar persoanelor peste 18 ani.
- La plasarea unei comenzi, se consideră plata efectuată. Nu se cunoaște încă exact data livrării. Atunci când comanda este confirmată, suma ajunge la vânzător, statusul comenzii fiind finalizat.

Simplificări implementate:

- Prețuri fixe pentru transport: național (25 lei) și internațional (50 lei)
- Fotografiile produselor și ale utilizatorilor nu sunt gestionate în baza de date
- Plățile efective nu sunt gestionate intern, ci doar statusul lor
- În caz de retur, vânzătorul trebuie să creeze un anunț nou pentru produs

2. Structura și Inter-relaționarea tabelor

Entități principale:

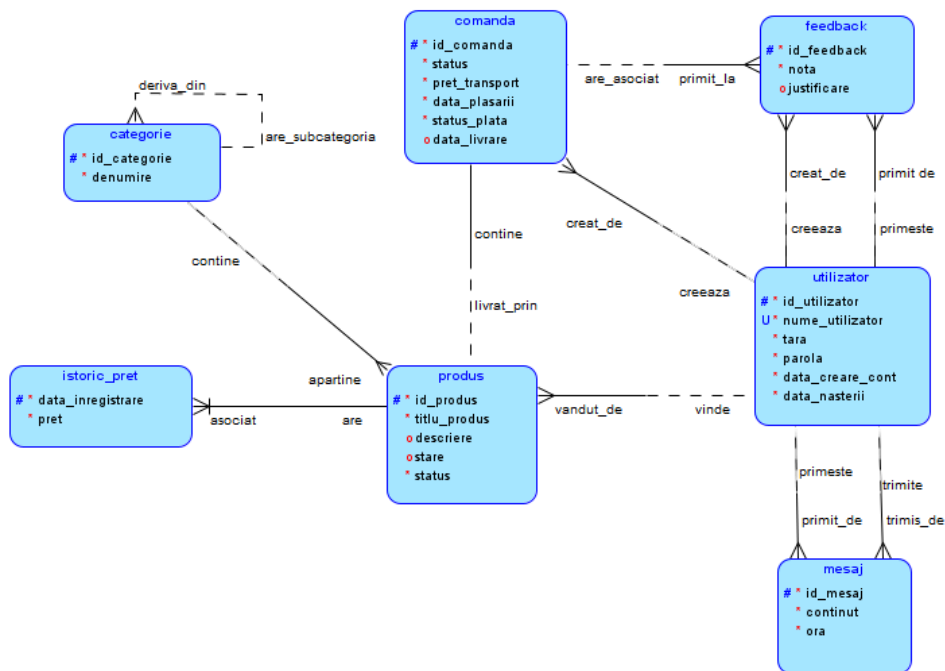
1. **Utilizator** - gestionează conturile utilizatorilor
2. **Categorie** - organizare ierarhică a categoriilor de produse
3. **Produs** - anunțurile de vânzare
4. **Istoric_pret** - evidența modificărilor de preț pentru fiecare produs
5. **Comanda** - tranzacțiile de vânzare-cumpărare

6. **Feedback** - sistemul de rating între utilizatori

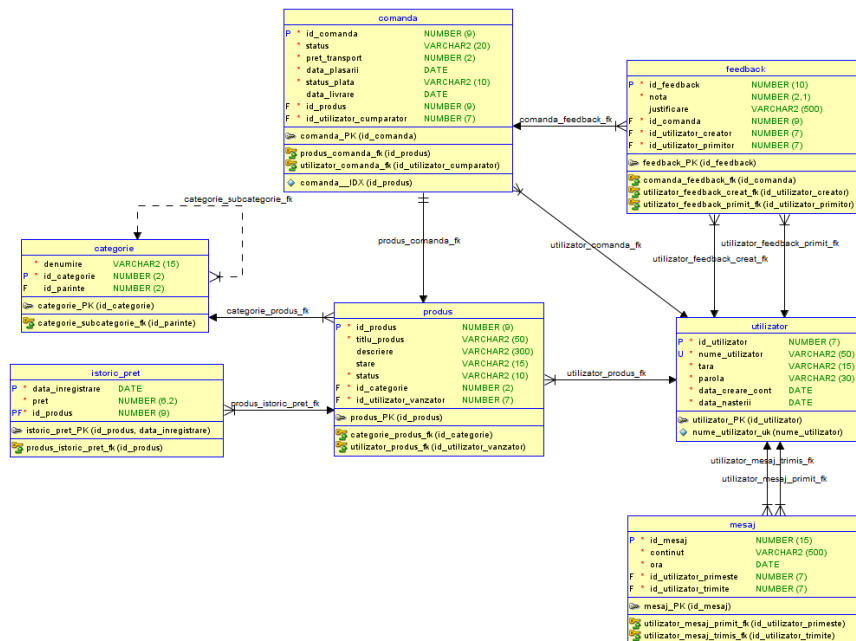
7. **Mesaj** - comunicarea între utilizatori

Relații între entități:

- **Utilizator** → **Produs** (1:N): Un utilizator poate vinde multiple produse, dar un produs aparține unui singur vânzător
- **Utilizator** → **Comanda** (1:N): Un utilizator poate face multiple comenzi ca și cumpărător
- **Utilizator** → **Mesaj** (1:N × 2): Un utilizator poate trimite și primi multiple mesaje
- **Utilizator** → **Feedback** (1:N × 2): Un utilizator poate crea și primi multiple feedback-uri
- **Categorie** → **Categorie** (1:N reflexivă): O categorie poate avea subcategorii (structură ierarhică)
- **Categorie** → **Produs** (1:N): O categorie poate conține multiple produse
- **Produs** → **Istoric_pret** (1:N): Un produs poate avea multiple intrări în istoric (negocieri)
- **Produs** → **Comanda** (1:1): Un produs poate fi vândut o singură dată
- **Comanda** → **Feedback** (1:N): O comandă poate genera multiple feedback-uri (de la cumpărător și vânzător)



-Model logic-



-Model relațional-

Aspecte legate de normalizarea tabelor:

- Respectă prima formă normală (1NF) deoarece valorile sunt atomice (atribute nedivizibile, nu avem liste de valori în cadrul unei singure coloane). Exemplu: *stare* din tabela *produs* conține o singură valoare din domeniu ('nou', 'foarte_buna', etc.), nu liste de stări.
- Respectă a doua formă normală (2NF) pentru că toate coloanele non-cheie depind de cheia primară. Exemplu: În tabela *utilizator*: attributele *nume_utilizator*, *parola*, *tara*, *data_creatie_cont*, *data_nasterii* sunt unic identificate prin *id_utilizator*.
- Respectă a treia formă normală (3NF) fiindcă nu există dependențe tranzitive între atribute. Nu există situații în care un atribut non-cheie să depindă de alt atribut non-cheie. Exemplu: În tabela *comanda*, *id_produkt* și *id_utilizator_cumparator* sunt chei externe independente, nu există o dependență tranzitivă între ele.
- Schema nu se află în BCNF deoarece există tabele în care apar dependențe funcționale în care determinantul nu este super-cheie, cum este cazul tabelii *comanda*, unde *id_produkt* determină atribute ale comenzii (relația 1-1), deși cheia primară este *id_comanda*.

Schema este proiectată în **3NF**, iar deși nu respectă BCNF, acest lucru este acceptabil deoarece păstrarea dependențelor funcționale non-cheie este necesară pentru logica aplicației și pentru a evita descompuneri care ar complica inutil modelul.

Fluxul unei comenzi:

1. Cumpărătorul plasează comanda și efectuează plata (status: plasat)
2. Vânzătorul confirmă și expediază produsul (status: confirmat/expediat)
3. Cumpărătorul primește produsul și confirmă (status: livrat)
4. Plata este transferată vânzătorului (status_plata: finalizat)
5. Ambele părți pot lăsa feedback

3. Justificarea Tipurilor de Date și Dimensiunilor

Identificatori numerici (ID-uri): Toate ID-urile folosesc NUMBER cu dimensiuni calculate pentru scalabilitate: id_utilizator NUMBER(7) permite până la 10 milioane utilizatori, id_produc NUMBER(9) permite miliarde de anunțuri, iar id_mesaj NUMBER(15) permite volum extrem de mare de conversații. Dimensiunile sunt alese generos pentru a acomoda creșterea pe termen lung.

Prețuri și note: pret este NUMBER(6,2) pentru maxim 9999.99 lei cu precizie de bani (conform cerințelor: până la 2000 lei), iar nota este NUMBER(2,1) pentru rating 0.0-5.0 cu o zecimală (ex: 4.5 stele).

Câmpuri text variabile: VARCHAR2 este folosit pentru flexibilitate, cu dimensiuni optimizate: Dimensiunile permit cazuri uzuale fără risipă de stocare.

4. Constrângeri și Justificarea Lor

PRIMARY KEY (PK): Toate tabelele au PK-uri generate automat prin secvențe și trigger BEFORE INSERT, eliminând erorile umane și asigurând unicitate absolută. ISTORIC_PRET are PK compusă (id_produc, data_inregistrare) pentru a permite evoluția prețurilor în timp.

FOREIGN KEY (FK): 11 FK-uri asigură integritatea referențială: produs referă categorie și utilizator (vânzător), comanda referă produs și utilizator (cumpărător), feedback referă comanda și doi utilizatori (creator/primitor), mesaj referă doi utilizatori (expeditor/destinatar), istoric_pret referă produs, iar categorie se auto-referă pentru ierarhie (id_parinte).

UNIQUE: nume_utilizator trebuie să fie unic pentru autentificare fără confuzii. Comanda are UNIQUE INDEX pe id_produc implementând regula critică de business: **un produs = o singură vânzare**, prevenind supravânzarea aceluiași articol fizic.

NOT NULL: Aplicată pe câmpuri esențiale: toate ID-urile și FK-urile (integritate), nume_utilizator/parola/data_nasterii (cont funcțional), titlu_produc/status (anunț valid), continut_mesaj (comunicare), nota_feedback (rating obligatoriu), statusuri și date_comenzi (tracking complet). Câmpuri opționale: descriere_produc, justificare_feedback, data_livrare (până la confirmare), id_parinte_categorie (NULL pentru categorii principale).

CHECK - Validări format: denumire categorie (min 3 litere, doar alfanumeric), titlu_produc (min 5 caractere), nume_utilizator (min 5 caractere, fără spații, fără whitespace la capete), parola (min 8 caractere pentru securitate).

CHECK - Validări domeniu: Standardizare valori pentru consistență: stare produs în ('nou', 'foarte_buna', 'buna', 'acceptabila', 'rea'), status produs în ('activ', 'inactiv'), status comandă în (7 valori de la 'plasat' la 'returnat'), pret_transport în (25, 50) pentru național/internațional, status_plata în ('efectuat', 'finalizat', 'restituit'), tara în (13 țări Europa Est/Centrală), nota BETWEEN 0 AND 5, pret BETWEEN 0 AND 2000.

CHECK - Validări logică: data_livrare trebuie NULL sau mai mare decât data_plasarii (logică temporală: produsul nu poate fi livrat înainte de plasarea comenzii).

DEFAULT: Valori automate pentru ușurință: tara = 'romania' (utilizatori majoritari), data_creare_cont/data_plasarii/ora/data_inregistrare = SYSDATE (timestamp automat), stare = 'acceptabila' (neutru), status produs = 'activ' (vizibil automat), status comandă = 'plasat' (inițial), pret_transport = 25 (național, cel mai frecvent), status_plata = 'efectuat' (plata instant la plasare).

TRIGGERE - Protecție audit trail: 4 trigger BEFORE UPDATE blochează modificarea timestamp-urilor critice (data_plasarii, data_inregistrare, ora, data_creare_cont), prevenind coruperea istoricului.

TRIGGER - Validare vârstă: trg_utilizator_data_nasterii_trg (BEFORE INSERT/UPDATE) verifică că utilizatorul are 18-100 ani, asigurând conformitate legală pentru plăți online (limita inferioară) și filtrând date invalide (limita superioară).

TRIGGER - Business logic: produs_inactiv_dupa_comanda_trg (AFTER INSERT pe COMANDA) setează automat status = 'inactiv' pentru produsul comandat, implementând regula că fiecare anunț devine invizibil automat după vânzare, fără intervenție manuală.

Concluzii

Baza de date pentru aplicația Vinted este proiectată să susțină un marketplace cu accent pe: integritatea tranzacțiilor prin constrângeri și trigger de validare, transparența prețurilor prin istoricul complet al negocierilor, reputația utilizatorilor prin sistemul de feedback bilateral, scalabilitate prin dimensionarea adecvată a tipurilor de date